A Comparison of Heuristic and Metaheuristic Methods for solving the Traveling Salesman Problem

1st Daniel Schafhäutle
Computational and Data Science
FHGR
Chur, Switzerland
daniel.schafhautle@stud.fhgr.ch

2nd Joshua Kohler
Computational and Data Science
FHGR
Chur, Switzerland
joshua.kohler@stud.fhgr.ch

3rd Gian-Luca Lanfranchi
Computational and Data Science
FHGR
Chur, Switzerland
gianluca.lanfranchi@stud.fhgr.ch

***Abstract*—The traveling salesman problem (TSP) is an NP hard combinatorial optimization problem, that asks for the shortest possible route that visits a set of cities and returns to the starting city. Solving TSPs exactly using brute-force methods quickly becomes computationally infeasible as the number of cities increases. However, there exists a number of heuristic and metaheuristic algorithms, that can find reasonably good solutions in a much shorter time. This paper presents a comparison of five such methods, namely: Simulated Annealing, Tabu Search, 2-Opt, Iterated Local Search and a Genetic Algorithm. Algorithms are evaluated using benchmark TSP instances of sizes n = [10, 25, 50, 100, 200, 500] with k = 30 instances each. We preform this experiment with two types of instances: one with uniformly sampled cities and one with clustered cities. They are then compared based on the solution quality measured as the gap to the optimal solution obtained by the Concorde TSP solver, and the time required to find the solution. The goal of this paper is to compare the strengths and weaknesses of these algorithms, that are the basis of many state of the art methods for solving TSPs and provide a starting point for further research into this area.**

I. Introduction

The traveling salesman problem (TSP) is a classic combinatorial optimization problem that asks for the shortest possible route that visits a set of cities and returns to the origin city. More formally, given a complete Graph $G = (V, E)$, where $V = \{v_1, v_2, \ldots, v_n\}$ is the set of nodes representing the cities, and $d(v_i, v_j)$ is the distance associated with the edge between each pair of nodes $v_i, v_j \in V$. A solution to the TSP is an ordering of cities $(v_{\pi(1)}, v_{\pi(2)}, \ldots, v_{\pi(n)})$ that minimizes the total tour length:

$$\left(\sum_{i=1}^{n-1} d(v_{\pi(i)}, v_{\pi(i+1)}) \right) + d(v_{\pi(n)}, v_{\pi(1)}) \tag{1}$$

Where $\pi(i)$ determines the index of the city that is visited at step i [1].

This is an NP-hard combinatorial optimization problem. That means, that there is no known polynomial time algorithm, that can solve it optimally for all instances [1]. It is so hard to solve, because the number of possible tours is $(n-1)!/2$ and grows with $\mathcal{O}(n!)$ [1]. This makes it infeasible to brute-force a solution for problems with more than a few nodes. A problem with 100 nodes has 10^{155} possible tours, to check all of them naively would take longer than the age of the universe. Because we don't gave that kind of time, we need to find ways to quickly find solutions to TSPs that are *good enough*, meaning that they are not optimal, but sufficiently close to the optimal solution. However, for a lot of applications a solution that is not quite optimal is all that is needed. Heuristics are used to find these near optimal solutions within reasonable time.

II. Related Work

- Mention that exact solvers like Concorde use cutting-plane methods and branch-and-bound to find the true mathematical optimum.
- Briefly reference classic literature (like the Glover paper mentioned in your README) regarding how local search and metaheuristics revolutionized combinatorial optimization.

III. Materials and Methods

A. 2-Opt

B. Simulated Annealing

C. Tabu Search

D. Iterated Local Search

Algorithm 1 Iterated Local Search

$s_0 = \text{GenerateInitialSolution}()$
$s^* = \text{LocalSearch}(s_0)$
$s_{\text{best}} = s^*$
repeat
  $s' = \text{Perturbation}(s^*)$
  $s^{*\prime} = \text{LocalSearch}(s')$
  if $\text{Cost}(s^{*\prime}) < \text{Cost}(s_{\text{best}})$ **then**
    $s_{\text{best}} = s^{*\prime}$
  end if
  if $\text{Cost}(s^{*\prime}) < \text{Cost}(s^*)$ **then**
    $s^* = s^{*\prime}$
  end if
until termination condition met (e.g., iterations $= 200$)
return s_{best}

E. Genetic Algorithm

IV. Experiments and Results

Detail exactly how you conducted the experiment so someone else could replicate it

A. Setup

1) Dataset Generation:

2) Benchmark:

3) Hardware / Environment:

B. Results

- Performance Metrics
- Plots

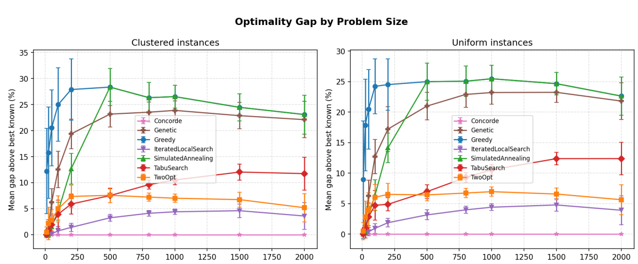

Fig. 1. Optimality gap of algorithms for different sizes and distributions

Probably split Figure 1 in half, so that it is nicely visible in single column layout.

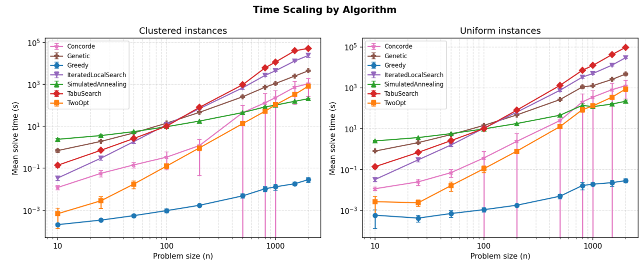

Fig. 2. Time scaling of algorithms for different sizes and distributions.

V. Discussion

Trade off analysis. Speed vs Quality.

Best all rounder, fast but sloppy, accurate but expensive...

Explain why the results look like that. Why is one algorithm better able to handle clusters than another. etc.

VI. Conclusion

Summarize core finding.

suggest future directions

- Other algorithms. Hyperparameter tuning.

References

[1] M. R. Garey and D. S. Johnson, *Computers and Intractability*. wh freeman New York, 2002, vol. 29.

VII. The First Appendix